ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

ДОКЛАД

к защите курсового проекта на тему:

«Проектирование и разработка модуля регистрации и обработки программных ошибок»

Выполнил: студент группы ИСиП/23-1 Цоколов Даниил Дмитриевич

Руководитель: Крючков Виталий Владимирович

Регламент выступления — 7–8 минут. Метки слайдов соответствуют презентации.

(слайд 1)

Уважаемые члены комиссии! Вашему вниманию представляется курсовой проект на тему «Проектирование и разработка модуля регистрации и обработки программных ошибок», выполненный на материале серверной части локального мессенджера AirGrap — веб-приложения на платформе ASP.NET Core с базой данных PostgreSQL.

(слайд 2)

Актуальность работы обусловлена тем, что программные ошибки при разработке неизбежны, а их некорректная обработка превращается в угрозу безопасности: подробные сообщения об исключениях и стеки вызовов раскрывают злоумышленнику внутреннее устройство системы; отсутствие журналирования не позволяет обнаружить и расследовать попытки несанкционированного доступа; неуправляемые ошибки нарушают доступность

сервиса. Федеральные законы № 149-ФЗ «Об информации, информационных технологиях и о защите информации» и № 152-ФЗ «О персональных данных», а также ГОСТ Р 51583-2014 относят регистрацию событий безопасности к функциям защиты информации в автоматизированных системах.

(слайд 3)

Целью курсового проекта является проектирование и разработка модуля регистрации и обработки программных ошибок для серверной части мессенджера AirGrap. Для её достижения поставлены шесть задач: изучить теоретические основы обработки ошибок; рассмотреть механизмы платформы C#/.NET; проанализировать серверную часть AirGrap; определить требования и спроектировать модуль; реализовать модуль на языке C#; провести тестирование и сформулировать рекомендации по доработке.

(слайд 4)

Объектом исследования является процесс обработки программных ошибок в серверной части веб-приложения, предметом — методы и программные средства регистрации и обработки ошибок на платформе ASP.NET Core с СУБД PostgreSQL. Методы исследования: анализ литературы и стандартов, системный анализ, объектно-ориентированное проектирование и функциональное тестирование.

(слайд 5)

Серверная часть реализована на языке C# на платформе .NET 10 с применением ASP.NET Core, драйвера Npgsql 10 и СУБД PostgreSQL с расширениями pgcrypto для хеширования паролей и pg_timetable для фоновых заданий. Для документирования и тестирования интерфейса используется Scalar с поддержкой OpenAPI.

(слайд 6)

Архитектура серверной части слоистая: уровень контроллеров принимает HTTP-запросы и выполняет первичную валидацию; уровень сервисов содержит бизнес-логику регистрации и аутентификации; уровень доступа к данным создаёт источник NpgsqlDataSource по параметрам из конфигурационного

файла; нижний уровень — СУБД PostgreSQL. Разрабатываемый модуль охватывает уровни сервисов и контроллеров, а также файловый журнал.

(слайд 7)

Анализ процессов регистрации и аутентификации показал, что эти операции насыщены точками возникновения ошибок: валидация входных данных в контроллере; ограничения базы данных — дубликат логина порождает ошибку SQLSTATE 23505, пустые обязательные поля — 23502; отказы инфраструктуры. При этом регистрация и вход — самые употребимые и самые атакуемые операции мессенджера, поэтому их ошибки обязаны фиксироваться в журнале.

(слайд 8)

Перейду к результатам практической части.

(слайд 9)

К модулю предъявлены функциональные требования: классификация ошибок СУБД по кодам SqlState, журналирование необработанных ошибок, безопасные HTTP-ответы без раскрытия внутренних деталей, первичная валидация входных данных и понятные описания ошибок; а также нефункциональные — расширяемость и минимальные зависимости. Ядром модуля стала иерархия пользовательских исключений: базовый класс Exception и три наследника — UserAlreadyExistsException с кодом 23505 и свойством Login, NullInputFieldException с кодом 23502 и RegistrationFailedException для прочих ошибок СУБД.

(слайд 10)

Компонент журналирования реализован статическим классом Logger с методом AddLog, дописывающим текст события в конец файла Logs/log.txt. Анализ выявил и ограничения этого решения: в записи нет метки времени, уровня события, типа исключения и стека вызовов, а часть ошибок сервиса выводится в консоль, минуя журнал. Эти ограничения учтены в рекомендациях.

(слайд 11)

Обработка ошибки построена на двух уровнях. На уровне сервиса блоки перехвата с фильтрами when классифицируют PostgresException по коду SqlState и превращают его в доменное исключение. На уровне контроллера доменные исключения отображаются на коды HTTP-ответов: занятый логин — 400 BadRequest, пустое обязательное поле — 409 Conflict, любые прочие ошибки — 500 с обобщённым сообщением, а перед возвратом ответа текст исключения фиксируется в журнале. Внутренние детали ошибки клиенту не передаются.

(слайд 12)

Модуль прошёл функциональное тестирование по восьми сценариям через интерактивную документацию Scalar: проверялись код ответа, тело ответа и содержимое журнала. Все результаты совпали с ожидаемыми: повторная регистрация существующего логина даёт HTTP 400 с понятным сообщением; остановленная СУБД — HTTP 500 с обобщённым сообщением и записью в журнал. Требования Ф1–Ф5 подтверждены.

(слайд 13)

По итогам анализа и тестирования сформулированы рекомендации: подключить глобальный обработчик исключений UseExceptionHandler; расширить формат журнала меткой времени, уровнем и стеком, в перспективе — перейти на ILogger или Serilog; журналировать в сервисе через Logger и вести аудит успешных операций; реализовать исключения аутентификации; согласовать коды HTTP-ответов; задействовать таблицу аудита log на уровне СУБД и исправить неработающий скрипт triggers.sql; вынести параметры подключения в пользовательские секреты; унифицировать наименования каталогов и файлов.

(слайд 14)

Таким образом, задачи курсового проекта решены полностью, цель достигнута. Разработанный модуль встроен в работающее серверное приложение: ошибки регистрируются и обрабатываются предсказуемо, внутренняя информация не раскрывается, журнал позволяет анализировать нештатные ситуации.

(слайд 15)

Спасибо за внимание! Готов ответить на ваши вопросы.